

Vulnerability Assessment Report

AuSecurity - External Vulnerability Assessment Report

AuSecurity

63 Paddington St
Paddington NSW 2021, Australia

Flinders University
Sturt Road, Bedford Park SA
5042

September 18, 2026

Report Version: 1.0

Table of Contents

List of Figures	4
1 Document Control	5
1.1 Team	5
1.2 List of Changes	5
2 Executive Summary	6
2.1 Overview	6
2.2 Identified Vulnerabilities	7
3 Methodology	9
3.1 Objective	9
3.2 Scope	9
3.3 User Accounts and Permissions	10
4 Findings	11
C1: Unauthenticated osTicket installer wizard publicly reachable	11
H1: Domain has no SPF, DKIM, DMARC, MX or MTA-STS records - full email spoofability	13
H2: WordPress XML-RPC endpoint exposed with dangerous methods enabled	15
M1: Self-signed TLS certificate on primary website	17
M2: osTicket session cookie set without Secure, HttpOnly, or SameSite attributes	19
M3: SSH management port exposed to the public Internet on all three VPS	21
M4: All HTTP responses lack modern security headers	23
M5: All-in-One WP Migration plugin exposed with known-vulnerable code path history	25
M6: Backend origin webserver exposed cleartext on TCP/8080	27
M7: WordPress /wp-includes/ directory listing enabled	29
M8: WordPress user enumeration via REST API and author archive	30

L1: Weak TLS cipher suites offered on multiple HTTPS listeners	32
I1: Contact form honeypot field is present but not enforced	34
I2: WordPress REST API exposes 135 routes including editing and batch endpoints	35
I3: WordPress site exposes readme.html, license.txt, and default admin content	37
5 Disclaimer	39
A Appendix	40
A.1 Static Appendix Section	40

List of Figures

Figure 1 - Distribution of identified vulnerabilities	8
---	---

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report issued to the client.	Sep 18, 2026

2 Executive Summary

2.1 Overview

AuSecurity engaged ClawSec to perform an authorized, full-scope, external vulnerability assessment of its Internet-facing systems. The assessment took place on 18 September 2026 and covered the primary website, the marketing/contact site, the WordPress blog, and the customer support (osTicket) system, all hosted on three dedicated OVH virtual servers in Singapore. Testing followed a non-destructive methodology (passive reconnaissance, controlled enumeration, and manually verified vulnerability probing) and did not use denial-of-service, credential brute forcing, phishing, or lateral-movement techniques.

Overall, AuSecurity's public footprint is small, but the environment shows clear signs of not having been hardened after initial deployment. Fifteen issues were confirmed and are grouped as follows:

Severity	Count
Critical	1
High	2
Medium	7
Low	2
Informational	3

Most urgent issue. The customer support system at `ticket.ausecurity.best` is still in an unfinished installation state, and its setup wizard is reachable from anywhere on the Internet without any password. In practice, a stranger on the Internet can complete the installation and become the helpdesk administrator, which would give them ownership of the support system used to communicate with customers. This should be closed within hours, not days.

Two additional high-priority issues. First, the main website at `ausecurity.best` is currently serving a self-signed certificate, which causes every visitor's browser to show a full-screen "your connection is not private" warning even though a valid Let's Encrypt certificate has already been issued and only needs to be deployed. For a business that sells governance, risk and compliance services, this warning materially damages first impressions and trust. Second, the `ausecurity.best` domain publishes no email authentication records (SPF, DKIM, DMARC) and no mail routing (MX), which means (a) anyone on the Internet can send convincing emails that appear to come from `@ausecurity.best`, and (b) the advertised business contact `info@ausecurity.best` cannot actually receive mail. Both are inexpensive to fix.

Remaining findings. The medium- and lower-severity items are common configuration issues (missing HTTP security headers, WordPress user enumeration, exposed backend origin ports, WordPress XML-RPC weaknesses, missing session-cookie flags, weak TLS cipher choices, default WordPress artefacts, an unenforced honeypot on the contact form). Individually they are modest, but taken together they indicate an environment that has not yet been hardened to a production baseline.

Business context. AuSecurity's service offering is GRC consulting and penetration testing. The findings in this report are the kind of issues AuSecurity itself would raise with a client, so remediating them quickly protects both the business and its professional reputation.

Recommended next steps (in order):

1. Take the osTicket installer offline immediately (firewall rule or webserver block) and either complete or remove the installation.
2. Deploy the existing Let's Encrypt certificate on the primary website and add the two missing subdomains to it.
3. Publish SPF, DKIM, DMARC, and (once a mail provider is chosen) MX and MTA-STS records for `ausecurity.best`.
4. Add the standard HTTP security headers, close the direct-to-origin port 8080 exposure, and rename the default WordPress `admin` account.
5. Restrict SSH management access at the network layer to known administrator IPs.

A full technical description of every finding, along with reproduction evidence and prioritized remediation guidance, is provided in the Findings section of this report.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Unauthenticated osTicket installer wizard publicly reachable	11
H1	8.2	Domain has no SPF, DKIM, DMARC, MX or MTA-STS records - full email spoofability	13
H2	7.2	WordPress XML-RPC endpoint exposed with dangerous methods enabled	15
M1	6.8	Self-signed TLS certificate on primary website	17
M2	5.9	osTicket session cookie set without Secure, HttpOnly, or SameSite attributes	19
M3	5.6	SSH management port exposed to the public Internet on all three VPS	21
M4	5.4	All HTTP responses lack modern security headers	23
M5	5.3	All-in-One WP Migration plugin exposed with known-vulnerable code path history	25
M6	5.3	Backend origin webserver exposed cleartext on TCP/8080	27
M7	5.3	WordPress /wp-includes/ directory listing enabled	29
M8	5.3	WordPress user enumeration via REST API and author archive	30
L1	3.7	Weak TLS cipher suites offered on multiple HTTPS listeners	32
I1	0.0	Contact form honeypot field is present but not enforced	34
I2	0.0	WordPress REST API exposes 135 routes including editing and batch endpoints	35
I3	0.0	WordPress site exposes readme.html, license.txt, and default admin content	37

Vulnerability Overview

In the course of this penetration test **1 Critical**, **2 High**, **8 Medium**, **1 Low** and **3 Info** vulnerabilities were identified:

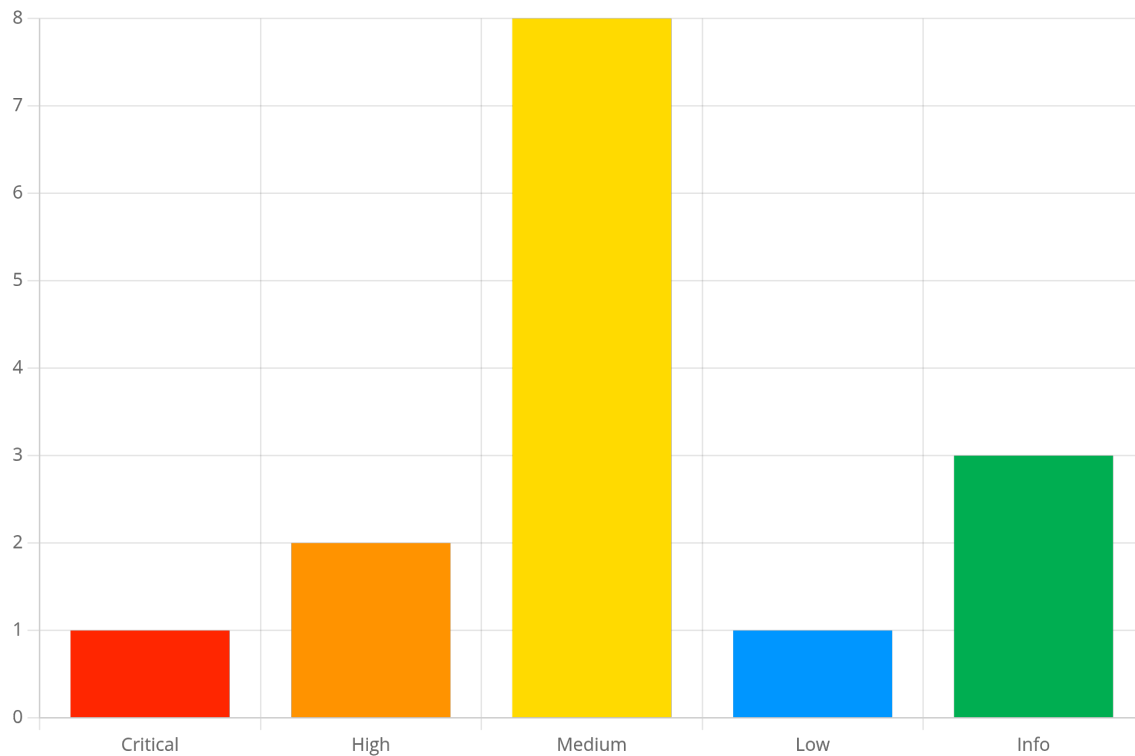


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasible, seamless and cost-affective such approach is.

3.2 Scope

This engagement was an **authorized, full-scope, external (Internet-facing) vulnerability assessment** of AuSecurity's public assets. All testing was performed from the public Internet against assets that were confirmed to belong to AuSecurity before any probing began. Scope authorization was obtained by the client publishing the required `AuthorizedCode` HTTP response header on the primary public-facing website (<https://ausecurity.best/>), which was verified at 2026-09-18T02:38:21Z.

In-scope assets:

System	Description
ausecurity.best	Root domain / primary marketing website
www.ausecurity.best	www alias of the primary website
contact.ausecurity.best	Contact site with FastAPI contact-form backend
blog.ausecurity.best	WordPress-based company blog
ticket.ausecurity.best	osTicket customer support portal
51.79.166.148	OVH VPS hosting the primary/contact web stack
51.79.166.209	OVH VPS hosting the osTicket application
51.79.167.208	OVH VPS (blog / auxiliary)

Eight web endpoints were tested across these hosts (HTTPS on TCP/443 for all four subdomains, plus HTTP on TCP/8080 on both origin webserver and HTTP on TCP/80 on the blog).

Explicitly out of scope (third-party infrastructure):

- Spaceship (DNS registrar / hosted DNS)
- OVH (underlying cloud/VPS infrastructure)
- Google Fonts, Automattic Gravatar (external CDN dependencies referenced by the blog)

Methodology: Passive reconnaissance (DNS/WHOIS/Certificate Transparency), light-active enumeration (subdomain, port, and service discovery), and non-destructive vulnerability probing across all confirmed in-scope assets. All commands were reviewed by the ClawSec orchestrator before

execution. No destructive techniques (denial-of-service, credential brute forcing, MFA bypass, phishing, malware deployment, or lateral movement) were used. Every finding in this report was manually verified prior to inclusion.

Limitations honestly acknowledged:

- The Nuclei baseline template scan (7,888 templates across 8 URLs) completed roughly 54% of planned requests within its 30-minute time budget before being stopped. Targeted manual assessment was subsequently performed on all high-value paths, so no known high-value area was left uncovered, but a longer automated scan may surface additional low-severity noise.
- The WPScan paid vulnerability database token was not available, so WordPress plugin/theme CVE mapping was performed manually against publicly known issues rather than against the commercial WPScan feed.
- The osTicket application was in an incomplete / pending-installation state, so runtime application-level testing (authenticated features, authorization checks, business-logic testing) could not be performed against a fully-installed application.
- No password guessing or brute-force testing was performed against SSH, WordPress admin, osTicket, or any other login surface.

3.3 User Accounts and Permissions

No credentials or user accounts were provided. This engagement was performed entirely from an unauthenticated, external attacker perspective.

4 Findings

C1: Unauthenticated osTicket installer wizard publicly reachable	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	• https://ticket.ausecurity.best/setup/install.php • http://51.79.166.209:8080/setup/install.php
References	• https://docs.osticket.com/en/latest/Getting%20Started/Installation.html • https://owasp.org/www-project-application-security-verification-standard/ • https://cwe.mitre.org/data/definitions/1188.html

Overview

The osTicket helpdesk at `ticket.ausecurity.best` is in an unfinished installation state, and its installer wizard is publicly reachable without any authentication at `/setup/install.php`. Every route on the application (root, the `/scp/login.php` admin console, the `/api/http.php` API) redirects unauthenticated visitors straight to the installer, which contains no CSRF token, IP restriction, or authentication challenge. The same exposure is reproducible directly against the origin at `http://51.79.166.209:8080/setup/install.php`.

Details

Reproduction / evidence:

- GET `https://ticket.ausecurity.best/` returns 302 -> `/setup/install.php`.
- GET `https://ticket.ausecurity.best/setup/install.php` returns 200 OK with an HTML page titled "Installing osTicket v1.18.4".
- The installer form (`<form method="post" action="install.php">`) contains only a single hidden field `s=prereq` and a submit button. No CSRF token, nonce, or authentication challenge is present.
- The same behaviour is reproduced directly against the origin at `http://51.79.166.209:8080/setup/install.php`, meaning the exposure is not only at the reverse-proxy edge.
- Server-side details disclosed on the installer page and in HTTP response headers: `Server: nginx/1.22.1`, `X-Powered-By: PHP/8.2.33`, `osTicket v1.18.4`, and the full PHP extension inventory (IMAP / APCu missing).
- The application has been in this pending-setup state since at least 2026-09-14 (TLS certificate issue date), so it has been publicly waiting to be finished for days.

Business impact:

An unauthenticated attacker anywhere on the Internet can walk through the installer, connect the application to an attacker-controlled (or attacker-created) database, and create the initial administrator account. osTicket runs PHP on the same webserver, so the attacker then holds an authenticated application-admin position from which further web-shell / arbitrary-file upload paths become available (osTicket admins can install language packs, plugins, and edit templates - all of which have led to remote code execution in past deployments).

In the worst case this is a full application takeover of AuSecurity's helpdesk system, which typically stores customer support tickets, uploaded attachments, and information users paste into tickets. It also enables convincing phishing sent *from* `ticket.ausecurity.best`, using AuSecurity's real brand and TLS certificate.

Recommendation

1. **Immediately** restrict access to `/setup/` on both the nginx reverse proxy (port 443) and directly on the Apache origin (port 8080). Use an IP allowlist, HTTP basic-auth, or a firewall rule limited to the deployment team.
2. Complete or abandon the osTicket install. Once installation is finished, delete the `setup/` directory from the web root - the vendor explicitly instructs administrators to do this.
3. Do not expose the origin (`51.79.166.209:8080`) directly to the public Internet. Bind Apache to `127.0.0.1:8080` or block the port at the host firewall so all traffic must transit the reverse proxy.
4. After install, remove or hide the `Server` header (`ServerTokens Prod + ServerSignature Off`) and disable `expose_php` in `php.ini` to suppress the `X-Powered-By: PHP/8.2.33` disclosure.
5. Add HSTS, CSP, `X-Frame-Options`, and set the `PHPSESSID` cookie with `Secure; HttpOnly; SameSite=Lax` (see the related "Missing HTTP security headers" and "Insecure session cookie" findings).

H1: Domain has no SPF, DKIM, DMARC, MX or MTA-STS records - full email spoofability

Score	8.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:H/A:N
Target	ausecurity.best (entire domain)
References	• https://datatracker.ietf.org/doc/html/rfc7208 • https://datatracker.ietf.org/doc/html/rfc6376 • https://datatracker.ietf.org/doc/html/rfc7489 • https://datatracker.ietf.org/doc/html/rfc8461 • https://dmarc.org/ • https://cwe.mitre.org/data/definitions/940.html

Overview

`ausecurity.best` publishes no MX record, no SPF record, no DKIM selector records, no DMARC record, and no MTA-STS policy. As a result, (1) the advertised business contact address `info@ausecurity.best` cannot reliably receive email on this domain, and (2) any third party on the Internet can send email that appears to originate from `@ausecurity.best` and it will not be rejected on authentication grounds by receiving mail servers.

Details

Reproduction / evidence:

- `dig MX ausecurity.best +short -> empty.`
- `dig TXT ausecurity.best +short -> no v=spf1 ... string.`
- `dig TXT _dmarc.ausecurity.best +short -> no v=DMARC1 ... string.`
- `dig TXT _mta-sts.ausecurity.best +short -> empty.`
- DKIM selector probes for `default`, `selector1`, `selector2`, `google`, `k1`, `mail`, `smtp`, `mandrill`, `mailgun` at `_domainkey.ausecurity.best` all returned `NXDOMAIN`.
- The published contact page on `contact.ausecurity.best` still lists `info@ausecurity.best` as the sole business email address. Because no MX is published, mail sent there bounces at the sender's side (some providers deliver to an A-record fallback, others do not).

Business impact:

- **Phishing.** An attacker can send convincingly-forged emails "from" `admin@ausecurity.best`, `billing@ausecurity.best`, `vinh@ausecurity.best`, and so on to AuSecurity's customers, prospects, or partners. Gmail, Microsoft 365 and other receivers do apply DMARC-derived reputation, but with no SPF/DKIM/DMARC published they cannot reject on authentication - only on content signals and static domain reputation, which are weak for a young domain.

- **Business communication loss.** Anyone trying to email `info@ausecurity.best` may see a permanent-failure bounce, causing loss of inbound leads.
- **Brand damage.** A single successful phishing wave sent to a customer will attribute damage to AuSecurity, even though AuSecurity's own servers were never involved.

Recommendation

1. Decide how the business will receive mail. If Google Workspace or Microsoft 365 is used, publish the vendor's MX records for `ausecurity.best`.
2. Publish a strict SPF record. If no server should ever send mail as `@ausecurity.best`, use `v=spf1 -all`. Otherwise, enumerate every sending source (for example: `v=spf1 include:_spf.google.com -all`).
3. Enable DKIM in whichever provider is used and publish the provider's selector TXT record.
4. Publish a starting DMARC policy: `v=DMARC1; p=none; rua=mailto:vinh@ausecurity.best; ruf=mailto:vinh@ausecurity.best; fo=1; adkim=s; aspf=s` and monitor the aggregate reports for at least two weeks. Then move to `p=quarantine` and finally `p=reject`.
5. Publish MTA-STS (`_mta-sts.ausecurity.best` + `mta-sts.ausecurity.best`) once a mail provider is in place.

H2: WordPress XML-RPC endpoint exposed with dangerous methods enabled

Score	7.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:L/I:N/A:L
Target	https://blog.ausecurity.best/xmlrpc.php
References	• https://make.wordpress.org/core/2015/04/28/xml-rpc-and-remote-publishing/ • https://blog.sucuri.net/2014/03/more-than-162000-wordpress-sites-used-for-distributed-denial-of-service-attack.html • https://cwe.mitre.org/data/definitions/441.html

Overview

`xmlrpc.php` on the WordPress blog responds to POST requests with a full `system.listMethods` reply containing 86 methods, including `pingback.ping`, `wp.getUsers`, `wp.getAuthors`, and `metaWeblog.getUsersBlogs`. `pingback.ping` in particular is a well-known SSRF and reflected-DDoS primitive: an attacker asks the WordPress server to make an HTTP request to any URL of their choice, which can be aimed at internal network addresses (SSRF), used to amplify a DDoS against a third-party target, or used to fingerprint the internal side of the network.

Details

Reproduction / evidence:

- `curl -X POST -H "Content-Type: text/xml" -d "<methodCall><methodName>system.listMethods</methodName></methodCall>" https://blog.ausecurity.best/xmlrpc.php` -> XML response listing 86 methods including `pingback.ping`, `pingback.extensions.getPingbacks`, `wp.getUsers`, `wp.getAuthors`, `wp.getUserInfo`, `metaWeblog.getUsersBlogs`, and `system.multicall`.
- A test pingback request pointed at `https://example.com/` and `https://blog.ausecurity.best/?p=1` (a non-existent post) returned a fault, which indicates the endpoint accepted the call and attempted to process it. Pingback is therefore live and usable once a real target post ID is supplied.
- `system.multicall` amplifies password-guessing attempts against `wp.getUsersBlogs`: an attacker can submit many username/password pairs in a single HTTP request, bypassing per-request rate limits.

Business impact:

- **Server-side request forgery.** The WordPress host can be coerced into probing internal network resources (`http://127.0.0.1:8080/`, cloud metadata endpoints, other private hosts on the same VLAN).

- **DDoS reflection.** A large number of hosts calling `pingback.ping` at a chosen target overloads that target while attributing traffic to WordPress installations - including this one.
- **Credential-guessing amplification.** `system.multicall` combined with `wp.getUsersBlogs` allows hundreds of login attempts per HTTP request, defeating naive rate limiting.

Recommendation

1. Disable XML-RPC entirely if it is not used. In `functions.php` or a mu-plugin:
`add_filter( 'xmlrpc_enabled', '__return_false' );`.
2. Block at the webserver so no XML-RPC request even reaches PHP. For Apache:

```
<Files "xmlrpc.php">  
    Require all denied  
</Files>
```

3. If XML-RPC is required for a specific integration (for example the Jetpack app), IP-allowlist it to that integration only.
4. Also disable pingback specifically: `add_filter('xmlrpc_methods', function($m)
{ unset($m['pingback.ping']); unset($m['pingback.extensions.getPingbacks']); return
$m; });`.

M1: Self-signed TLS certificate on primary website	
Score	6.8 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N
Target	• https://ausecurity.best/ • https://www.ausecurity.best/ • https://contact.ausecurity.best/ • 51.79.166.148:443
References	• https://ssl-config.mozilla.org/ • https://datatracker.ietf.org/doc/html/rfc5280 • https://cwe.mitre.org/data/definitions/295.html

Overview

The primary marketing website ausecurity.best (and its [www](https://www.ausecurity.best) and [contact](https://contact.ausecurity.best) aliases) presents a self-signed TLS certificate issued by [CN=ausecurity.best](#) to [CN=ausecurity.best](#), valid from 2026-09-14 to 2027-09-14. Every mainstream browser will display a full-screen warning ("Your connection is not private / NET::ERR_CERT_AUTHORITY_INVALID") before letting a visitor click through. A valid Let's Encrypt certificate was already issued for this hostname on 2026-09-07 (Certificate Transparency ID 29414811460 on crt.sh) - the correct certificate exists but has simply not been deployed to nginx.

Details

Reproduction / evidence:

- `openssl s_client -connect ausecurity.best:443 -servername ausecurity.best` returns a certificate with `Issuer=CN=ausecurity.best` (i.e. self-signed) instead of the expected Let's Encrypt R3 / E1 / E2 issuer.
- The certificate's Subject Alternative Name is only `DNS:ausecurity.best`, so `https://www.ausecurity.best/` and `https://contact.ausecurity.best/` additionally trigger a hostname-mismatch warning even after the browser is instructed to trust the self-signed certificate.
- The nginx configuration on `51.79.166.148` is therefore either pointing at a leftover self-signed pair generated during initial provisioning, or the Let's Encrypt renewal client wrote the certificate to a path nginx is not reading.

Business impact:

1. *Trust and lead conversion.* Every prospective customer is greeted by a "your connection is not private" browser warning. For a company that sells GRC consulting and penetration testing, this materially damages first impressions and lead conversion.
2. *Security posture.* Users are conditioned to click through certificate warnings, which teaches them to accept the same warning during a genuine machine-in-the-middle attack. It also disables the browser's principal defence against DNS/BGP hijacking of [ausecurity.best](#).
3. *Downstream integrations.* Secondary services relying on certificate validation (HSTS preload, monitoring, curl-based integrations, some link scanners) will refuse to talk to the site at all.

Recommendation

1. Deploy the existing Let's Encrypt certificate for `ausecurity.best` (from crt.sh id 29414811460) into the nginx configuration on `51.79.166.148`. Verify that `ssl_certificate` and `ssl_certificate_key` in the nginx server block reference the LE-issued file paths (usually `/etc/letsencrypt/live/ausecurity.best/fullchain.pem` and `.../privkey.pem`).
2. Issue additional certificates (or add SANs) that cover `www.ausecurity.best` and `contact.ausecurity.best`. `certbot certonly --nginx -d ausecurity.best -d www.ausecurity.best -d contact.ausecurity.best` in one call is the simplest path.
3. Confirm renewal is automated (`systemctl status certbot.timer` or equivalent).
4. After the correct certificate is deployed, add `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` to force TLS on future visits.

M2: osTicket session cookie set without Secure, HttpOnly, or SameSite attributes

Score	5.9 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:L/A:N
Target	• https://ticket.ausecurity.best/ • http://51.79.166.209:8080/
References	• https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/1004.html • https://cwe.mitre.org/data/definitions/614.html • https://cwe.mitre.org/data/definitions/1275.html

Overview

Every response from `ticket.ausecurity.best` sets `Set-Cookie: PHPSESSID=<value>` without the `Secure`, `HttpOnly`, or `SameSite` attributes. Once the osTicket installation is completed and administrators / support agents begin authenticating, this makes session hijacking trivial through any of several common vectors.

Details

Reproduction / evidence:

- Observed header: `Set-Cookie: PHPSESSID=03f049da4755a8987e7b2a10278ed602; path=/`.
- No `Secure` flag: the cookie will be sent over cleartext HTTP as soon as any request hits `http://ticket.ausecurity.best/` or `http://51.79.166.209:8080/` (both of which respond and would collect the cookie).
- No `HttpOnly`: any XSS on the domain lets JavaScript read `document.cookie` and exfiltrate the session id.
- No `SameSite`: the cookie is sent on all cross-site requests, enabling CSRF (harmful once the app is set up and users are authenticated).

Business impact:

Currently the risk is latent because the osTicket install has not yet been completed, but once administrators and support agents start authenticating, session hijacking becomes trivial through any of the vectors above. Session hijacking gives full agent or admin access to the helpdesk queue, containing customer support tickets, uploaded files, and often sensitive customer data.

Recommendation

Configure PHP session cookies in `php.ini` (or via `.htaccess`) globally on the ticket host:

```
session.cookie_secure = 1  
session.cookie_httponly = 1  
session.cookie_samesite = "Lax"
```

Additionally, in the nginx reverse-proxy configuration, rewrite `Set-Cookie` headers to append the flags as a defence in depth:

```
proxy_cookie_flags PHPSESSID secure httponly samesite=lax;
```

M3: SSH management port exposed to the public Internet on all three VPS

Score	5.6 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:L
Target	• 51.79.166.148:22 • 51.79.166.209:22 • 51.79.167.208:22
References	• https://www.cisecurity.org/benchmark/debian_linux • https://www.ncsc.gov.uk/ • https://cwe.mitre.org/data/definitions/284.html

Overview

All three OVH VPS hosts accept TCP connections to port 22 (SSH) from any source IP on the Internet. The SSH daemon itself is modern (OpenSSH 9.2p1) and offers strong key-exchange and cipher choices, but two legacy MAC algorithms (`hmac-sha1` and `hmac-sha1-etm@openssh.com`) are still offered for backward compatibility. Host-key fingerprints are unique across the three VPS (no VM cloning). No login attempts or brute forcing were performed.

Details

Reproduction / evidence:

- Nmap NSE `ssh2-enum-algos` output shows: KEX includes `sntrup761x25519`, `curve25519`, ECDHE-nistp256/384/521, DH-group14/16/18 (strong). Ciphers: `chacha20-poly1305`, `aes128/192/256-ctr`, `aes128/256-gcm` (all strong). MACs: `umac-64/128` (both raw and -etm), `hmac-sha2-256/512` (both raw and -etm), plus the two legacy `hmac-sha1` variants noted above.
- Banners are identical across all three hosts: `SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u10`.
- Host-key fingerprints:
 - 51.79.166.148: ECDSA `9a:bd:49:00:5b:b0:59:65:53:a3:6c:86:3a:ae:ed:9a`, ED25519 `1d:ef:e7:0b:cc:5c:b8:c2:95:24:16:84:46:f1:d3:0e`
 - 51.79.166.209: ECDSA `54:64:22:67:4b:a1:50:93:c9:2e:f5:c4:e7:8b:44:5e`, ED25519 `27:fa:c8:43:03:c8:08:b9:01:63:17:fb:28:df:69:2d`
 - 51.79.167.208: ECDSA `24:6e:a6:ee:62:0b:a9:2c:73:c1:9a:e3:bf:e1:2c:fe`, ED25519 `e7:1a:00:70:5a:64:4f:f8:fe:b4:29:19:ef:ba:a1:a6`

Business impact:

Public SSH exposure is the default of most OVH images, and OpenSSH 9.2 is well-hardened, so the direct risk from the SSH daemon itself is low. The risk is behavioural: the wider the audience of possible authenticators, the more the deployment depends entirely on password/key strength for security, and the more log noise (bots) obscures real events.

Recommendation

1. Restrict SSH to specific management IPs at the OVH firewall or the local host firewall (`iptables` / `nftables` / `ufw`). If a bastion or VPN exists, use it.
2. Enforce key-only authentication (`PasswordAuthentication no`, `PermitEmptyPasswords no`, `PermitRootLogin prohibit-password`).
3. Disable the two legacy MACs in `/etc/ssh/sshd_config` (or a drop-in in `/etc/ssh/sshd_config.d/`):

```
MACs -hmac-sha1,hmac-sha1-etm@openssh.com
```

4. Enable `fail2ban` or the OVH-side network protection to slow brute-force attempts.
5. Consider changing the listening port to a non-default value to reduce noise (defence in depth; do not rely on it as a security control).

M4: All HTTP responses lack modern security headers

Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	• https://ausecurity.best/ • https://www.ausecurity.best/ • https://contact.ausecurity.best/ • http://51.79.166.148:8080/ • https://blog.ausecurity.best/ • http://blog.ausecurity.best/ • https://ticket.ausecurity.best/ • http://51.79.166.209:8080/
References	• https://owasp.org/www-project-secure-headers/ • https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security • https://cwe.mitre.org/data/definitions/693.html

Overview

Every in-scope web response is missing every one of the commonly-recommended HTTP security headers: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, and the `Cross-Origin-*`-Policy family. Additionally, the `Server` header reveals the exact webserver version (nginx/1.22.1, nginx/1.25.5, Apache/2.4.68) on every response, and the osTicket endpoints add `X-Powered-By: PHP/8.2.33`.

Details

Reproduction / evidence:

`curl -sIL` on each endpoint returned the following, uniformly:

- `Strict-Transport-Security`: missing
- `Content-Security-Policy`: missing
- `X-Frame-Options`: missing
- `X-Content-Type-Options`: missing
- `Referrer-Policy`: missing
- `Permissions-Policy`: missing
- `Cross-Origin-Opener-Policy` / `-Embedder-Policy` / `-Resource-Policy`: missing
- `Server`: exposed (nginx or Apache with exact version)
- `X-Powered-By`: exposed on osTicket (PHP/8.2.33)

Business impact:

- Without HSTS, a first-time visitor navigating to `http://ausecurity.best/` (or any subdomain served on port 80) is vulnerable to SSL-stripping and downgrade attacks.
- Without `X-Frame-Options` or a `frame-ancestors` CSP directive, every page can be framed inside another site, enabling clickjacking of any form (contact form, future login, osTicket admin console once installed).

- Without `X-Content-Type-Options: nosniff`, browsers may execute a response as JavaScript even when the declared MIME type is not JS. The risk is small on static pages, but rises sharply once user-uploaded content appears (WordPress uploads folder, osTicket attachments).
- The version banners let an attacker target published CVEs precisely; suppressing them raises the cost of exploitation.

Recommendation

For each site, add the following headers at the reverse-proxy layer (nginx) so that a single change protects all backends:

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;
add_header Content-Security-Policy "default-src 'self'; img-src 'self' data: https://
secure.gravatar.com; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-
src 'self' https://fonts.gstatic.com; script-src 'self'" always;
server_tokens off;
```

For osTicket, also set `expose_php = Off` in `php.ini`.

Do **not** enable HSTS until the self-signed certificate finding above is fixed (the browser will otherwise get pinned to a broken certificate).

M5: All-in-One WP Migration plugin exposed with known-vulnerable code path history

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://blog.ausecurity.best/wp-content/plugins/all-in-one-wp-migration/ • https://blog.ausecurity.best/wp-content/ai1wm-backups/
References	• https://wordpress.org/plugins/all-in-one-wp-migration/#developers • https://nvd.nist.gov/vuln/detail/CVE-2023-40004 • https://nvd.nist.gov/vuln/detail/CVE-2024-31111 • https://cwe.mitre.org/data/definitions/284.html

Overview

The WordPress blog at `blog.ausecurity.best` has All-in-One WP Migration v7.109 installed and the backup storage folder `wp-content/ai1wm-backups/` is reachable from the public Internet (HTTP 200). The `web.config` file inside that folder is served with a 200 response. The plugin's `readme.txt` is publicly readable and discloses the exact installed version.

Details

Reproduction / evidence:

- GET `https://blog.ausecurity.best/wp-content/plugins/all-in-one-wp-migration/readme.txt` -> 200 OK, Stable tag: 7.109.
- GET `https://blog.ausecurity.best/wp-content/ai1wm-backups/` -> 200 OK (folder exists and is publicly reachable).
- GET `https://blog.ausecurity.best/wp-content/ai1wm-backups/web.config` -> 200 OK (356 bytes).
- No actual `.wppress` backup files were present in the folder at the time of testing.

The plugin's public history includes multiple broken-access-control CVEs where authenticated (or in some versions unauthenticated) users could trigger export/import operations. Even if the current 7.109 build is patched, the plugin's presence combined with a publicly-listable backup directory means that any future backup written to that folder becomes trivially downloadable and contains the entire site (posts, users' password hashes in `wp_users`, secrets in `wp_options`, and often the equivalent of `wp-config.php` constants).

Business impact:

If an administrator ever runs a backup (manually or on schedule), the resulting `.wppress` file becomes downloadable by anyone on the Internet. A `.wppress` backup contains the entire WordPress database and file tree, including hashed user credentials, session salts, API keys, and any customer data stored in the site. The directory exposure also allows an attacker to discover backup filenames via file enumeration and to poll periodically for new backups.

Recommendation

1. Block direct browser access to `/wp-content/ai1wm-backups/` and its children on the Apache origin:

```
<Directory /var/www/blog/wp-content/ai1wm-backups>  
    Require all denied  
</Directory>
```

(or an nginx equivalent if a proxy is added).

2. If the plugin is not actively used for migrations, uninstall it. All-in-One WP Migration should generally not be left running on a production site.
3. Keep the plugin updated (7.109 was current at test time - verify against the vendor changelog).
4. Store backups off-site (S3, external SFTP), not under `wp-content/`.
5. Suppress `readme.txt` for all plugins:

```
<FilesMatch "^(readme|license|changelog)\.(txt|html|md)$">  
    Require all denied  
</FilesMatch>
```

M6: Backend origin webserver exposed cleartext on TCP/8080

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• http://51.79.166.148:8080/ • http://51.79.166.209:8080/
References	• https://owasp.org/www-project-application-security-verification-standard/ • https://cwe.mitre.org/data/definitions/668.html

Overview

Both AuSecurity webserver expose their internal origin daemons directly on TCP port 8080 with plain HTTP. The response body is byte-identical to the reverse-proxied version served on 443, but the origin advertises more precise server headers (revealing the true stack) and completely bypasses any TLS, HSTS, or WAF that the reverse proxy could impose. On `51.79.166.209:8080` the exposure also includes the same osTicket installer wizard described in the critical finding above, meaning any access control added to the reverse proxy would not protect against a direct attack.

Details

Reproduction / evidence:

- `curl http://51.79.166.148:8080/ -> 200 OK`, Server: `nginx/1.25.5`, body identical to `https://ausecurity.best/`.
- `curl http://51.79.166.209:8080/ -> 302 Found -> /setup/install.php`, Server: `Apache/2.4.68 (Debian)`, X-Powered-By: `PHP/8.2.33`. The osTicket installer is fully reachable here without HTTPS.
- Method probes (`OPTIONS`, `TRACE`, `PUT`, `DELETE`, `PATCH`) on `51.79.166.148:8080` all return `405`, so no WebDAV / trace-reflection weakness was found, but the endpoint is still reachable at all.
- Host-header confusion probes (`Host: admin.ausecurity.best`, `Host: internal.ausecurity.best`, `Host: 127.0.0.1`) on both origins returned identical responses to the default virtual host, so no hidden virtual hosts were discovered.

Business impact:

- All requests to `:8080` travel in cleartext. If an attacker can intercept traffic on the network path (uncommon for hosted VPS but not impossible), they can view or modify the request/response, including any session cookies or form data sent by anyone who knew the direct URL.
- The `:8080` endpoint reveals more precise stack versions (`nginx 1.25.5`, `Apache 2.4.68`, `PHP 8.2.33`) than the reverse proxy, aiding exploit targeting.
- Any hardening rule added to the nginx reverse proxy (WAF rules, rate limits, IP allowlists, CSP or HSTS injection, request-size caps) is completely bypassable by talking to `:8080` directly.

Recommendation

1. Bind the origin webserver to `127.0.0.1:8080` (loopback only) so they cannot be reached from outside the host. In the Apache/nginx origin config, change the listen directive: `Listen 127.0.0.1:8080` (Apache) / `listen 127.0.0.1:8080;` (nginx).
2. If for operational reasons the origin must remain reachable over the network, restrict access at the host firewall (ufw / nftables / OVH firewall panel):

```
iptables -I INPUT -p tcp --dport 8080 ! -s <reverse-proxy-IP> -j DROP
```

Since the reverse proxy is on the same host, `-s 127.0.0.1` is the safest allowlist.

3. Remove version disclosure from the origin (`ServerTokens Prod`, `ServerSignature Off`, `expose_php=Off`).

M7: WordPress /wp-includes/ directory listing enabled

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/wp-includes/
References	• https://owasp.org/www-project-application-security-verification-standard/ • https://cwe.mitre.org/data/definitions/548.html

Overview

Requesting <https://blog.ausecurity.best/wp-includes/> returns an Apache-generated HTML index of the WordPress core includes directory (approximately 65 KB of file names). The other WordPress folders (`/wp-content/`, `/wp-content/plugins/`, `/wp-content/themes/`) return a 200 with an empty body (index.html present), which correctly hides their contents.

Details

Reproduction / evidence:

- `curl https://blog.ausecurity.best/wp-includes/` returns a `<title>Index of /wp-includes</title>` HTML page listing every file and subdirectory.
- Sensitive files inside `wp-includes/` (for example `version.php` if directly requestable, and JS/CSS files that leak semver via URL) become trivially discoverable.
- The server is Apache/2.4.68 (Debian); the default `Options Indexes` is still enabled for this folder.

Business impact:

Directory listings do not directly execute anything, but they let an attacker enumerate every file present, target version-specific weaknesses more accurately, and discover any file left behind (for example a stray backup, an old plugin dumped into the wrong path, or a developer's test script).

Recommendation

1. Remove `Options Indexes` from the Apache configuration for the site. In the `vhost` or `.htaccess`:

```
Options -Indexes
```

2. Optionally place an empty `index.html` in `wp-includes/` (as WordPress already does in the sibling folders) as a defence in depth.

M8: WordPress user enumeration via REST API and author archive

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://blog.ausecurity.best/?rest_route=/wp/v2/users • https://blog.ausecurity.best/?author=1
References	• https://developer.wordpress.org/rest-api/frequently-asked-questions/ • https://cwe.mitre.org/data/definitions/200.html

Overview

Although the primary REST API route `/wp-json/wp/v2/users` returns 404, the equivalent query-string form `?rest_route=/wp/v2/users` returns a full JSON list of users, disclosing username, slug, display name, and gravatar URLs. Additionally, `?author=1` returns an HTTP 301 redirect to `/author/admin/`, confirming that user ID 1 exists and has the slug `admin`. Higher IDs return 404, confirming there is exactly one user.

Details

Reproduction / evidence:

- `curl "https://blog.ausecurity.best/?rest_route=/wp/v2/users"` returns JSON including `{"id":1,"name":"admin","slug":"admin","avatar_urls":{"...}}`.
- `curl -I "https://blog.ausecurity.best/?author=1"` returns 301 Location: `https://blog.ausecurity.best/index.php/author/admin/`.
- `curl -I "https://blog.ausecurity.best/?author=2..6"` all return 404.
- The default WordPress administrator username `admin` is in use.

Business impact:

An attacker knows with certainty that a valid administrator username is `admin`. This removes one factor from a two-factor guessing problem (username + password) and turns any successful password guess into an administrator takeover. Combined with the fact that `wp-login.php` is publicly reachable and there is no visible rate-limiting or CAPTCHA plugin, this materially raises the risk of successful credential guessing.

Recommendation

1. Rename the administrator account. Create a new administrator with a non-obvious username, log in as the new account, delete the `admin` user (or demote it to a role with no login), and reassign the sole post/page authorship.

2. Restrict the REST users endpoint. Add the following (or equivalent) to the theme's `functions.php` or a small mu-plugin:

```
add_filter( 'rest_endpoints', function( $endpoints ) {  
    if ( isset( $endpoints['/wp/v2/users'] ) ) unset( $endpoints['/wp/v2/users'] );  
    if ( isset( $endpoints['/wp/v2/users/(?P<id>[\d]+)'] ) ) unset( $endpoints['/wp/v2/  
users/(?P<id>[\d]+)'] );  
    return $endpoints;  
});
```

3. Block author enumeration in nginx/Apache:

```
if ($args ~ "^/?author=([0-9]+)") { return 403; }
```

4. Install a well-maintained login-protection plugin (Wordfence, Solid Security, Loginizer) to add lockouts, IP blocklists, and 2FA for `/wp-login.php`.

L1: Weak TLS cipher suites offered on multiple HTTPS listeners

Score	3.7 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://ausecurity.best:443 • https://www.ausecurity.best:443 • https://ticket.ausecurity.best:443
References	• https://ssl-config.mozilla.org/ • https://www.bsi.bund.de/ • https://cwe.mitre.org/data/definitions/326.html

Overview

`ausecurity.best` and `ticket.ausecurity.best` both offer TLS 1.2 cipher suites that fall outside the Mozilla "Intermediate" recommendation. The 64-bit truncated-MAC `AES*-CCM8` suites are considered weak by most compliance frameworks. `ticket.ausecurity.best` additionally accepts SHA-1-based `ECDHE-ECDSA-AES128-SHA` / `ECDHE-ECDSA-AES256-SHA`. `blog.ausecurity.best` was clean.

Details

Reproduction / evidence:

`ssllscan` output showed the following cipher suites enabled that should be disabled:

- `ausecurity.best:443` and `www.ausecurity.best:443`: `ECDHE-ECDSA-AES256-CCM8`, `ECDHE-ECDSA-AES128-CCM8`, `AES256-CCM8`, `AES128-CCM8` (64-bit MAC tag - non-compliant with PCI DSS 3.2.1+).
- `ticket.ausecurity.best:443`: the same CCM8 suites plus `ECDHE-ECDSA-AES256-SHA384`, `ECDHE-ECDSA-AES128-SHA256`, `ECDHE-ECDSA-AES256-SHA`, `ECDHE-ECDSA-AES128-SHA` (SHA-1 MAC variants).
- No SSLv2 / SSLv3 / TLS 1.0 / TLS 1.1 was offered - that portion of the config is modern.

Business impact:

No practical attack exists today against a properly-negotiated TLS 1.2 session using CCM8 or SHA-1 MAC (SLOTH-style downgrade requires specific conditions), but compliance frameworks flag these suites and they raise the exploitability of any future weakness found in truncated MACs. They also indicate that the server's TLS configuration was not tuned to a modern baseline.

Recommendation

Deploy a Mozilla "Intermediate" cipher list. For nginx:

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-
```

```
GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-  
POLY1305:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384;  
ssl_prefer_server_ciphers off;
```

For Apache/mod_ssl on the ticket host, use the equivalent `SSLCipherSuite` value from the Mozilla generator.

I1: Contact form honeypot field is present but not enforced	
Score	0.0 (Info)
Vector string	N/A
Target	https://contact.ausecurity.best/api/contact
References	https://owasp.org/www-project-application-security-verification-standard/

Overview

The FastAPI backend behind the contact form requires a field named `h` (present in the JSON schema, absent from the HTML form) which appears to be intended as a spam-bot honeypot. However, submissions in which `h` is filled with a non-empty value are accepted (HTTP 422 is only returned when the field is missing entirely, not when it is populated). The honeypot therefore does not filter automated form submissions.

Details

Reproduction / evidence:

- POST `{"h": ""}` (empty string) plus other required fields -> processed successfully.
- POST `{"h": "honeypot_test_value"}` (non-empty string) plus other required fields -> also processed successfully, with no rejection or additional error.
- The rest of the API is well-hardened: strict Pydantic validation, no reflected XSS, no SSTI, no CRLF injection, no mass-assignment (`extra_field`, `is_admin`, `role` all silently discarded), FastAPI docs (`/docs`, `/openapi.json`, `/redoc`) all return 404.

Business impact:

Automated form-spam bots will pass through the honeypot without being blocked, so the form will not be protected against low-effort spam. There is no direct security impact.

Recommendation

1. On the backend, reject any submission where `h` is non-empty (return `200 / 204` to the bot so it does not learn the filter is present).
2. Consider replacing or supplementing the honeypot with a CAPTCHA or an origin-IP rate limit for defence in depth.

I2: WordPress REST API exposes 135 routes including editing and batch endpoints

Score	0.0 (Info)
Vector string	N/A
Target	https://blog.ausecurity.best/?rest_route=/
References	https://developer.wordpress.org/rest-api/

Overview

The full WordPress REST API index (available via `?rest_route=`) lists 135 registered routes across the `oembed/1.0`, `wp/v2`, `wp-site-health/v1`, `wp-block-editor/v1`, and `wp-abilities/v1` namespaces. This includes `/batch/v1` (up to 25 sub-requests per request), plus template and global-style editing endpoints.

Details

Reproduction / evidence:

The full route list is captured in `evidence/T-03c/root.json`. Notable routes exposed:

- `/wp/v2/users` (via query-string workaround - see the separate user-enumeration finding)
- `/wp/v2/posts`, `/pages`, `/comments`, `/media`, `/templates`, `/template-parts`, `/global-styles` (editable when authenticated)
- `/wp/v2/settings`, `/wp/v2/themes`, `/wp/v2/plugins` (editable when authenticated)
- `/batch/v1` for compound requests
- Site Health checks under `/wp-site-health/v1`

All editing routes required authentication during testing - none allowed anonymous modification.

Business impact:

By itself this is normal WordPress behaviour. The concern is aggregate: once combined with the user-enumeration finding (an attacker has a confirmed valid username `admin`), the wide REST surface increases the impact of any credential compromise, because a single Application Password would grant the attacker full site edit through the REST API without ever needing to reach `/wp-admin/`.

Recommendation

1. Disable REST routes that are not needed. For example, if the site does not use the block editor for templates, block `/wp/v2/templates` and `/wp/v2/global-styles` via the `rest_endpoints` filter.

2. Disable Application Passwords if not used:

```
add_filter('wp_is_application_passwords_available', '__return_false');
```

3. Force authentication on the whole REST API for unauthenticated readers if the site does not need public REST access:

```
add_filter('rest_authentication_errors', function($result){  
    if (!empty($result)) return $result;  
    if (!is_user_logged_in()) return new WP_Error('rest_not_logged_in', 'You must be  
logged in.', ['status'=>401]);  
    return $result;  
});
```

I3: WordPress site exposes readme.html, license.txt, and default admin content

Score	0.0 (Info)	
Vector string	N/A	
Target	• https://blog.ausecurity.best/readme.html • https://blog.ausecurity.best/license.txt	• https://blog.ausecurity.best/?p=1 • https://blog.ausecurity.best/?page_id=2
References	https://wordpress.org/documentation/article/hardening-wordpress/	

Overview

The default WordPress artefacts (`readme.html`, `license.txt`, the "Hello world!" post at ID 1, and the "Sample Page" page at ID 2) are all still present and publicly reachable. The `readme.html` file discloses the exact WordPress core version.

Details

Reproduction / evidence:

- `curl https://blog.ausecurity.best/readme.html -> 200 OK`, 7.4 KB, includes a `Version 7.1.1` string.
- `curl https://blog.ausecurity.best/license.txt -> 200 OK`, 19.9 KB, standard GPL text (harmless but confirms WordPress).
- REST enumeration returned the "Hello world!" post and "Sample Page" page as the only content, plus an additional "Wow" page.
- The site is very newly deployed (first upload folder is `2026/09`, first content is stub content), which reinforces that it is a fresh install with default content still in place.

Business impact:

Low direct impact - the version disclosure is duplicated by the REST API and other channels. However, publishing default install artefacts signals a low-maintenance posture to attackers and lets automated scanners fingerprint WordPress and its exact core version instantly.

Recommendation

1. Delete `readme.html` and `license.txt` from the document root. Add a rewrite rule to block them if they will be recreated by an update:

```
<FilesMatch "^(readme|license)\.(txt|html)$">  
    Require all denied  
</FilesMatch>
```

2. Delete the default post ("Hello world!") and the default page ("Sample Page").
3. Publish real content or, if the blog is not yet ready for the public, restrict the entire host to authenticated access until launch.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerability assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section